00_preguntas_pequeñas

September 28, 2026

Connected to Python 3.13.3

[ ]:
```
import pandas as pd

#read data
basico = pd.read_csv("data/original/01_tablas_maestras/
 ↪Conjunto_datos_BASICO_AgroCebada2026.csv")
pro = pd.read_csv("data/original/01_tablas_maestras/
 ↪Conjunto_datos_PRO_AgroCebada.csv")


print("Basico data shape:", basico.shape)
print("Pro data shape:", pro.shape)
```

```
Basico data shape: (107666, 96)
Pro data shape: (47804, 20)
```

[ ]:
```
#columnas de cada tabla
basico_columns = basico.columns.tolist()
pro_columns = pro.columns.tolist()
ambas_columns = set(basico_columns).intersection(set(pro_columns))

print("Columnas en Basico:", basico_columns)
print("Columnas en Pro:", pro_columns)
print("Columnas en ambas tablas:", list(ambas_columns))
```

```
Columnas en Basico: ['ID_POLIGONO', 'fecha_captura', 'sensor', 'ndvi_promedio',
'ndvi_std', 'ndvi_max', 'ndvi_min', 'evi_promedio', 'evi_std', 'evi_max',
'evi_min', 'crc_promedio', 'crc_std', 'crc_max', 'crc_min', 'mcrc_promedio',
'mcrc_std', 'mcrc_max', 'mcrc_min', 'ndsvi_promedio', 'ndsvi_std', 'ndsvi_max',
'ndsvi_min', 'srndi_promedio', 'srndi_std', 'srndi_max', 'srndi_min',
'mra_promedio', 'mra_std', 'mra_max', 'mra_min', 'lai_promedio', 'lai_std',
'lai_max', 'lai_min', 'fapar_promedio', 'fapar_std', 'fapar_max', 'fapar_min',
'savi_promedio', 'savi_std', 'savi_max', 'savi_min', 'vi6t_promedio',
'vi6t_std', 'vi6t_max', 'vi6t_min', 'ndti_promedio', 'ndti_std', 'ndti_max',
'ndti_min', 'sti_promedio', 'sti_std', 'sti_max', 'sti_min', 'ndwi_promedio',
'ndwi_std', 'ndwi_max', 'ndwi_min', 'endwi_promedio', 'endwi_std', 'endwi_max',
'endwi_min', 'msi_promedio', 'msi_std', 'msi_max', 'msi_min', 'emsi_promedio',
'emsi_std', 'emsi_max', 'emsi_min', 'dswi2_promedio', 'dswi2_std', 'dswi2_max',
```

```
'dswi2_min', 'dswi3_promedio', 'dswi3_std', 'dswi3_max', 'dswi3_min',
'dswi4_promedio', 'dswi4_std', 'dswi4_max', 'dswi4_min', 'dswi5_promedio',
'dswi5_std', 'dswi5_max', 'dswi5_min', 'nddi_promedio', 'nddi_std', 'nddi_max',
'nddi_min', 'enddi_promedio', 'enddi_std', 'enddi_max', 'enddi_min',
'porcentaje_nubosidad']
Columnas en Pro: ['ID_POLIGONO', 'fecha_captura', 'sensor', 'ndvi_promedio',
'ndvi_std', 'ndvi_max', 'ndvi_min', 'evi_promedio', 'evi_std', 'evi_max',
'evi_min', 'lai_promedio', 'lai_std', 'lai_max', 'lai_min', 'msavi_promedio',
'msavi_std', 'msavi_max', 'msavi_min', 'porcentaje_nubosidad']
Columnas en ambas tablas: ['sensor', 'lai_promedio', 'evi_promedio', 'lai_std',
'ndvi_std', 'porcentaje_nubosidad', 'evi_max', 'ndvi_max', 'evi_min',
'ndvi_min', 'lai_max', 'fecha_captura', 'evi_std', 'lai_min', 'ID_POLIGONO',
'ndvi_promedio']
```

[ ]:
```
pro_landsat = pro[pro['sensor'] == 'Landsat']
pro_sentinel = pro[pro['sensor'] == 'Sentinel-2']

#drop null columns
#identify columns with all null values as null or empty columns or space as the␣
 ↪value
null_columns_landsat = pro_landsat.columns[pro_landsat.isnull().all()].tolist()
print("Null columns in Pro Landsat data:", null_columns_landsat)
pro_landsat = pro_landsat.drop(columns=null_columns_landsat)

null_columns_sentinel = pro_sentinel.columns[pro_sentinel.isnull().all()].
 ↪tolist()
print("Null columns in Pro Sentinel data:", null_columns_sentinel)
pro_sentinel = pro_sentinel.drop(columns=null_columns_sentinel)

print("Pro Landsat data shape:", pro_landsat.shape)
print("Pro Sentinel data shape:", pro_sentinel.shape)
```

```
Null columns in Pro Landsat data: ['ID_POLIGONO', 'fecha_captura', 'sensor',
'ndvi_promedio', 'ndvi_std', 'ndvi_max', 'ndvi_min', 'evi_promedio', 'evi_std',
'evi_max', 'evi_min', 'lai_promedio', 'lai_std', 'lai_max', 'lai_min',
'msavi_promedio', 'msavi_std', 'msavi_max', 'msavi_min', 'porcentaje_nubosidad']
Null columns in Pro Sentinel data: ['ID_POLIGONO', 'fecha_captura', 'sensor',
'ndvi_promedio', 'ndvi_std', 'ndvi_max', 'ndvi_min', 'evi_promedio', 'evi_std',
'evi_max', 'evi_min', 'lai_promedio', 'lai_std', 'lai_max', 'lai_min',
'msavi_promedio', 'msavi_std', 'msavi_max', 'msavi_min', 'porcentaje_nubosidad']
Pro Landsat data shape: (0, 0)
Pro Sentinel data shape: (0, 0)
```

[ ]:
```
basico_landsat = basico[basico['sensor'] == 'Landsat']
basico_sentinel = basico[basico['sensor'] == 'Sentinel-2']

#drop null columns
```

```
#identify columns with all null values as null or empty columns or space as the␣
 ↪value
null_columns_landsat = basico_landsat.columns[basico_landsat.isnull().all()].
 ↪tolist()
print("Null columns in Basic Landsat data:", null_columns_landsat)
basico_landsat = basico_landsat.drop(columns=null_columns_landsat)

null_columns_sentinel = basico_sentinel.columns[basico_sentinel.isnull().all()].
 ↪tolist()
print("Null columns in Basic Sentinel data:", null_columns_sentinel)
basico_sentinel = basico_sentinel.drop(columns=null_columns_sentinel)

print("Basic Landsat data shape:", basico_landsat.shape)
print("Basic Sentinel data shape:", basico_sentinel.shape)
```

```
Null columns in Basic Landsat data: ['ndvi_promedio', 'ndvi_std', 'ndvi_max',
'ndvi_min', 'evi_promedio', 'evi_std', 'evi_max', 'evi_min', 'crc_promedio',
'crc_std', 'crc_max', 'crc_min', 'mcrc_promedio', 'mcrc_std', 'mcrc_max',
'mcrc_min', 'ndsvi_promedio', 'ndsvi_std', 'ndsvi_max', 'ndsvi_min',
'srndi_promedio', 'srndi_std', 'srndi_max', 'srndi_min', 'mra_promedio',
'mra_std', 'mra_max', 'mra_min', 'lai_promedio', 'lai_std', 'lai_max',
'lai_min', 'fapar_promedio', 'fapar_std', 'fapar_max', 'fapar_min',
'savi_promedio', 'savi_std', 'savi_max', 'savi_min', 'ndti_promedio',
'ndti_std', 'ndti_max', 'ndti_min', 'sti_promedio', 'sti_std', 'sti_max',
'sti_min', 'ndwi_promedio', 'ndwi_std', 'ndwi_max', 'ndwi_min',
'endwi_promedio', 'endwi_std', 'endwi_max', 'endwi_min', 'msi_promedio',
'msi_std', 'msi_max', 'msi_min', 'emsi_promedio', 'emsi_std', 'emsi_max',
'emsi_min', 'dswi2_promedio', 'dswi2_std', 'dswi2_max', 'dswi2_min',
'dswi3_promedio', 'dswi3_std', 'dswi3_max', 'dswi3_min', 'dswi4_promedio',
'dswi4_std', 'dswi4_max', 'dswi4_min', 'dswi5_promedio', 'dswi5_std',
'dswi5_max', 'dswi5_min', 'nddi_promedio', 'nddi_std', 'nddi_max', 'nddi_min',
'enddi_promedio', 'enddi_std', 'enddi_max', 'enddi_min']
Null columns in Basic Sentinel data: ['vi6t_promedio', 'vi6t_std', 'vi6t_max',
'vi6t_min']
Basic Landsat data shape: (48626, 8)
Basic Sentinel data shape: (59040, 92)
```

[ ]:
```
landsat = basico_landsat
sentinel = basico_sentinel
planet = pro

fechas_landsat = landsat['fecha_captura'].unique()
fechas_sentinel = sentinel['fecha_captura'].unique()
fechas_planet = planet['fecha_captura'].unique()

#porcentaje de fechas  que coincide y fechas que no coincide
fechas_landsat_set = set(fechas_landsat)
```

```
fechas_sentinel_set = set(fechas_sentinel)
fechas_planet_set = set(fechas_planet)

all_fechas = fechas_landsat_set.union(fechas_sentinel_set).
 ↪union(fechas_planet_set)

intersection_landsat_sentinel = fechas_landsat_set.
 ↪intersection(fechas_sentinel_set)
intersection_landsat_planet = fechas_landsat_set.intersection(fechas_planet_set)
intersection_sentinel_planet = fechas_sentinel_set.
 ↪intersection(fechas_planet_set)

three_way_intersection = fechas_landsat_set.intersection(fechas_sentinel_set).
 ↪intersection(fechas_planet_set)


#print the results
print("Total dates", len(all_fechas))

print("Total unique dates in Landsat:", len(fechas_landsat_set))
print("Total unique dates in Sentinel:", len(fechas_sentinel_set))
print("Total unique dates in Planet:", len(fechas_planet_set))
print("Intersection of all three:", len(three_way_intersection))
```

```
Total dates 791
Total unique dates in Landsat: 392
Total unique dates in Sentinel: 301
Total unique dates in Planet: 311
Intersection of all three: 19
```

[ ]:
```
#entender preriodicidad y frecuencia de las fechas de captura de cada sensor

#planet
#reformat the date column from "%d/%m/%Y" to "%m/%d/%Y"
planet["fecha_captura"] = pd.to_datetime(planet["fecha_captura"], format="%d/%m/
 ↪%Y")
#frecuecnia de capturas planet
fechas_planet = planet["fecha_captura"].dropna().drop_duplicates()

yearly_frequency_planet = (
    fechas_planet.dt.year
    .value_counts()
    .sort_index()
)

monthly_frequency_planet = (
    fechas_planet.dt.to_period("M")
```

```
    .value_counts()
    .sort_index()
)

print("Capturas por año:")
print(yearly_frequency_planet)

print("\nCapturas por año-mes:")
print(monthly_frequency_planet)



initial_date_planet = planet['fecha_captura'].min()
final_date_planet = planet['fecha_captura'].max()
print("Initial date in Planet data:", initial_date_planet)
print("Final date in Planet data:", final_date_planet)

#landsat
landsat['fecha_captura'] = pd.to_datetime(landsat['fecha_captura'], format="%d/
 ↪%m/%Y")

# Frecuencia de capturas de landsat
fechas_landsat = landsat["fecha_captura"].dropna().drop_duplicates()

yearly_frequency_landsat = (
    fechas_landsat.dt.year
    .value_counts()
    .sort_index()
)

monthly_frequency_landsat = (
    fechas_landsat.dt.to_period("M")
    .value_counts()
    .sort_index()
)

print("Capturas por año:")
print(yearly_frequency_landsat)

print("\nCapturas por año-mes:")
print(monthly_frequency_landsat)

initial_date_landsat = landsat['fecha_captura'].min()
final_date_landsat = landsat['fecha_captura'].max()
print("Initial date in Landsat data:", initial_date_landsat)
print("Final date in Landsat data:", final_date_landsat)
```

```
#sentinel
sentinel['fecha_captura'] = pd.to_datetime(sentinel['fecha_captura'],␣
 ↪format="%d/%m/%Y")
frequency_sentinel = sentinel['fecha_captura'].value_counts().sort_index()
# Frecuencia de capturas de sentinel
fechas_sentinel = sentinel["fecha_captura"].dropna().drop_duplicates()

yearly_frequency_sentinel = (
    fechas_sentinel.dt.year
    .value_counts()
    .sort_index()
)

monthly_frequency_sentinel = (
    fechas_sentinel.dt.to_period("M")
    .value_counts()
    .sort_index()
)

print("Capturas por año:")
print(yearly_frequency_sentinel)

print("\nCapturas por año-mes:")
print(monthly_frequency_sentinel)
initial_date_sentinel = sentinel['fecha_captura'].min()
final_date_sentinel = sentinel['fecha_captura'].max()
print("Initial date in Sentinel data:", initial_date_sentinel)
print("Final date in Sentinel data:", final_date_sentinel)
```

```
Capturas por año:
fecha_captura
2025    311
Name: count, dtype: int64

Capturas por año-mes:
fecha_captura
2025-01    28
2025-02    24
2025-03    31
2025-04    29
2025-05    25
2025-06    18
2025-07    22
2025-08    27
2025-09    29
2025-10    26
2025-11    25
2025-12    27
```

```
Freq: M, Name: count, dtype: int64
Initial date in Planet data: 2025-01-01 00:00:00
Final date in Planet data: 2025-12-30 00:00:00
Capturas por año:
fecha_captura
2022    102
2023    107
2024     93
2025     90
Name: count, dtype: int64

Capturas por año-mes:
fecha_captura
2022-01     8
2022-02     6
2022-03     7
2022-04     8
2022-05     9
2022-06     9
2022-07     9
2022-08     9
2022-09     8
2022-10    10
2022-11    10
2022-12     9
2023-01     8
2023-02    10
2023-03     8
2023-04     9
2023-05     9
2023-06     9
2023-07     9
2023-08    10
2023-09    10
2023-10     9
2023-11     8
2023-12     8
2024-01     9
2024-02     8
2024-03     7
2024-04     7
2024-05     8
2024-06     8
2024-07     8
2024-08     8
2024-09     6
2024-10     8
2024-11     8
```

```
2024-12     8
2025-01     8
2025-02     6
2025-03     8
2025-04     8
2025-05     8
2025-06     7
2025-07     7
2025-08     8
2025-09     8
2025-10     7
2025-11     8
2025-12     7
Freq: M, Name: count, dtype: int64
Initial date in Landsat data: 2022-01-05 00:00:00
Final date in Landsat data: 2025-12-31 00:00:00
Capturas por año:
fecha_captura
2022    71
2023    71
2024    73
2025    86
Name: count, dtype: int64

Capturas por año-mes:
fecha_captura
2022-01    7
2022-02    5
2022-03    6
2022-04    6
2022-05    7
2022-06    6
2022-07    6
2022-08    6
2022-09    6
2022-10    6
2022-11    5
2022-12    5
2023-01    7
2023-02    5
2023-03    6
2023-04    6
2023-05    6
2023-06    6
2023-07    6
2023-08    6
2023-09    6
2023-10    5
```

```
2023-11    6
2023-12    6
2024-01    7
2024-02    5
2024-03    7
2024-04    6
2024-05    6
2024-06    6
2024-07    6
2024-08    6
2024-09    5
2024-10    6
2024-11    6
2024-12    7
2025-01    6
2025-02    5
2025-03    8
2025-04    8
2025-05    7
2025-06    8
2025-07    7
2025-08    7
2025-09    7
2025-10    7
2025-11    8
2025-12    8
Freq: M, Name: count, dtype: int64
Initial date in Sentinel data: 2022-01-01 00:00:00
Final date in Sentinel data: 2025-12-31 00:00:00
```